



Denoising Diffusion Probabilistic Models

🕒 생성일	@2025년 10월 8일 오후 4:25
🔖 태그	Done

Summary

1. Introduction

Diffusion probabilistic model

2. Background (Formula)

3. Diffusion models and denoising autoencoders

3.1 Forward process and L_T

3.2 Reverse process and $L_{1:T-1}$

3.3 Data Scaling, reverse process decoder

3.4 Simplified training objective

4. Experiments

4.1 Sample Quality

4.2 Reverse process parameterization and training objective ablation

4.3 Progressive coding

4.4 Interpolation

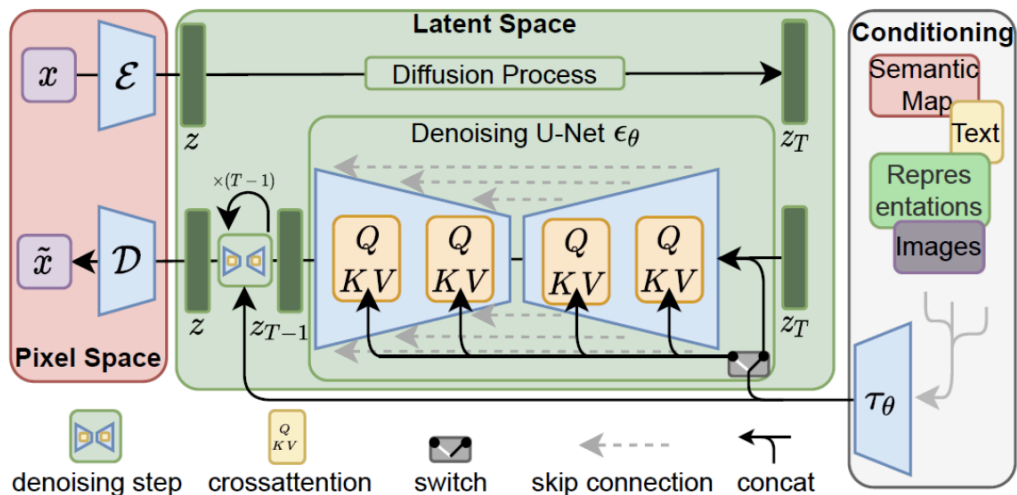
5. Related Work

Summary

1. Data에 random noise를 더해준 후 → 다시 noise를 제거 하는 과정을 학습하는 모델
2. **Forward process** : data에 noise를 추가하는 과정 ⇒ Markov Chain을 통해 점차적으로 noise를 더해 나감
3. **Reverse process** : Gaussian noise에서 시작하여 점차적으로 noise를 제거하는 과정

1. Introduction

- GAN, autoregressive models, normalizing flows, VAE는 striking image, audio sample을 합성
- GAN 뿐만 아니라 energy-based model이나 score matching과 같은 다른 접근법도 이제는 GAN과 필적할 수준의 결과를 도출해냄



- Diffusion 모델은 정의하기 간단하고 학습도 효율적이지만, 지금까지는 고품질 샘플을 생성할 수 있다는 증거가 없었음
 - sample의 품질 : GAN > Likelihood 기반 모델
 - 확률적 모델링 : GAN < Likelihood 기반 모델
- log-likelihood가 안좋다고 무조건 모델이 나쁜 것이 아니라는 재해석(in lossy compression 관점)

- Diffusion의 sampling = progressive decoding → autoregressive decoding의 일반화 및 새로운 해석을 제공
 - 이를 통해서 GAN이 가지지 못한 이론적 강점을 제시하였음

Diffusion probabilistic model

- Parameterized Markov chain을 이용해 유한한 시간 후 데이터와 일치하는 sample을 생성하기 위해 variational inference를 사용하여 학습됨
 - 이 chain의 transition은 diffusion process를 reverse 시키도록 학습되는데, 이 diffusion process는 data에 점진적으로 noise를 추가하는 Markov chain ⇒ sampling의 반대 방향으로 signal이 파괴될 때까지 계속됨

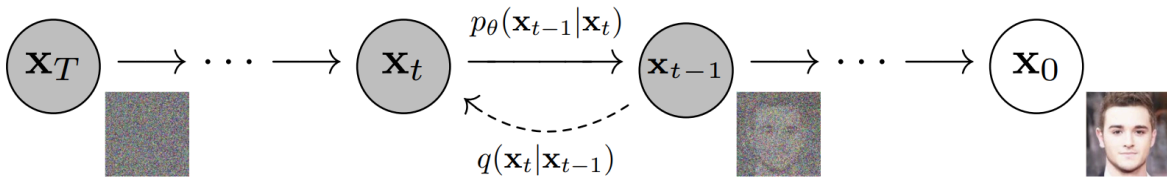


Figure 2: The directed graphical model considered in this work.

- diffusion model에서는 training 중 multi noise level에 대한 denoising score matching과 sampling 과정 중 annealed Langevin dynamics와 같다는 것을 보여줌
 - 이 논문에서는 이러한 구성 방식을 사용하여 best sample quality results를 획득
 - 이러한 sample quality에도 불구하고, diffusion model은 다른 log likelihood 기반의 모델들과 비교했을 때 경쟁력 있는 log likelihood를 갖지 않음

2. Background (Formula)

$$p_{\theta}(\mathbf{x}_0) := \int p_{\theta}(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$$

- diffusion model은 위의 형태를 가지는 latent variable models의 일종으로, $x_1, \dots, x_T$ 는 x_0 $q(x_0)$ 와 같은 dimensionality를 가지는 형태로 주어짐

- Joint distribution $p_\theta(x_{0:T})$ 는 reverse process라 하며, 이는 learned Gaussian transition을 가진 Markov Chain의 형태로 정의됨

$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)) \quad (1)$$

- diffusion model이 다른 종류의 latent variable model과 구별되는 점?
 - forward(diffusion) process라고 불리는 approximate posterior가 분산 일정 에 따라 data에 gaussian noise를 점진적으로 추가하는 Markov Chain에 고정 된다는 것

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad (2)$$

- Training은 negative log likelihood에 대한 일반적인 variational bound를 최적화함으로써 수행됨
- forward process variances는 reparameterization을 통해 학습될 수 있거나 상수로 유지될 수 있음

$$\mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L \quad (3)$$

- 표기법 $\alpha_t = 1 - \beta_t$ 와 $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ 를 사용하여, 다음의 결과를 얻음

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

- 효과적인 training은 stochastic gradient descent를 사용하여 L의 무작위 항목들을 최적화함으로써 가능하며, 더 나아가 L(3)을 다음과 같이 재작성함으로써 variance reduction의 개선이 이루어짐

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right] \quad (5)$$

- 위 수식은 KL divergence를 사용하여 $\mathbf{x}_0$ 에 조건을 부여했을 때 처리 가능한 forward process posteriors에 대해 $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ 를 직접 비교

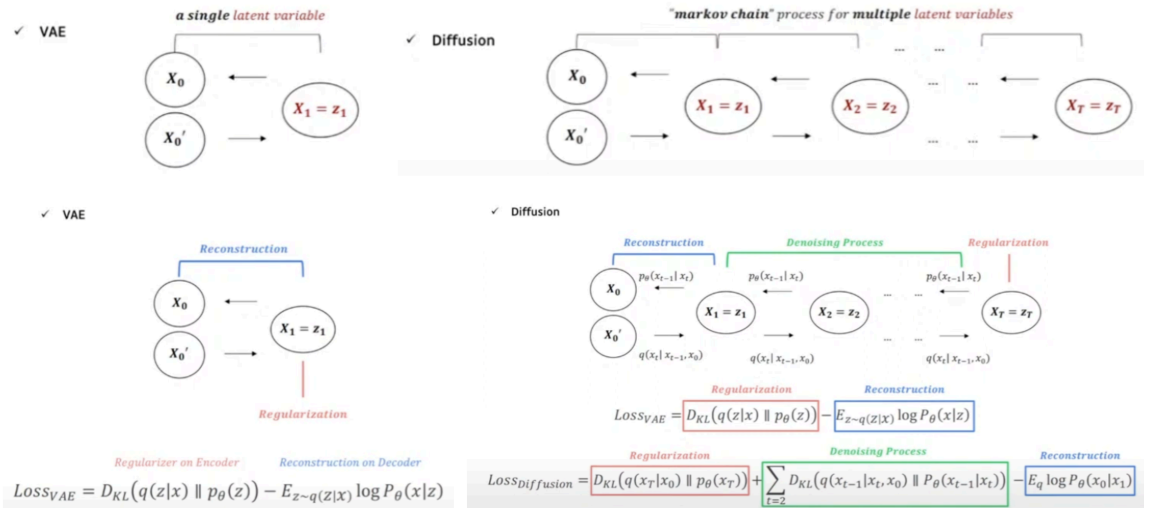
$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}), \quad (6)$$

$$\text{where } \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t \quad \text{and} \quad \tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t \quad (7)$$

- 결과적으로 (5) 수식의 모든 KL Divergence는 Gaussian 간의 비교이므로, high variance Monte Carlo estimate 대신 closed form의 표현식으로 Rao-Blackwellized 방식으로 계산할 수 있음

3. Diffusion models and denoising autoencoders

- VAE와 diffusion model 사이의 차이?



3.1 Forward process and L_T

- forward process의 variance β_T 가 reparameterization으로 학습 가능하다는 사실을 무시하고 대신 constant로 고정하였음
- 따라서 approximate posterior q에 learnable parameter가 없음 $\rightarrow L_T$ 는 훈련 도중 constant이며 무시할 수 있음

3.2 Reverse process and $L_{1:T-1}$

- $1 < t \leq T$ 에 대해 $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$ 에서 선택을 논의

1. $\Sigma_\theta(x_t, t) = \sigma_t^2 I$ 를 untrained time dependent constants로 설정

- 실험적으로, $\sigma_t^2 = \beta_t$ 와 $\sigma_t^2 = \tilde{\beta}t = \frac{1-\tilde{\alpha}t-1}{1-\tilde{\alpha}t}\beta_t$ 의 결과는 유사했음
- 첫 번째 선택은 $x_0 \sim \mathcal{N}(0, I)$ 에 최적이며, 두 번째는 x_0 를 결정론적으로 하나의 포인트로 설정하는 경우에 최적
 - 이는 coordinatewise unit variance를 가진 데이터에 대한 reverse process entropy의 upper과 lower에 해당하는 두 가지 극단적인 선택

2. 평균 $\mu_\theta(x_t, t)$ 를 나타내기 위해 L_t 의 분석에 기반한 specific parameterization을 제안

- $p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$ 와 함께, 다음과 같이 쓸 수 있음

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C \quad (8)$$

- 여기의 C는 constant이므로 θ 에 의존하지 않음
- 따라서, μ_θ 에 가장 직관적인 parameterization은 forward process posterior mean인 $\tilde{\mu}_t$ 를 예측하는 models임을 알 수 있음
 - 그러나 (4)를 $x_t(x_0, \epsilon) = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$ 으로 reparameterization하여 (8)를 더 확장 가능하며, 여기서 $\epsilon \sim \mathcal{N}(0, I)$ 를 따름
- 그리고 forward process posterior formula(7)을 적용

$$L_{t-1} - C = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \tilde{\mu}_t \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon), \frac{1}{\sqrt{\bar{\alpha}_t}} (\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \sqrt{1 - \bar{\alpha}_t} \epsilon) \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (9)$$

$$= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2\sigma_t^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t(\mathbf{x}_0, \epsilon) - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) - \mu_\theta(\mathbf{x}_t(\mathbf{x}_0, \epsilon), t) \right\|^2 \right] \quad (10)$$

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\nabla_\theta \left\ \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\ ^2$ 6: until converged	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

L_{simple} 을 사용한 최종적인 학습 과정이 Algorithm1에 나타남

3.3 Data Scaling, reverse process decoder

$$p_\theta(\mathbf{x}_0 | \mathbf{x}_1) = \prod_{i=1}^D \int_{\delta_-(x_0^i)}^{\delta_+(x_0^i)} \mathcal{N}(x; \mu_\theta^i(\mathbf{x}_1, 1), \sigma_1^2) dx \quad (13)$$

$$\delta_+(x) = \begin{cases} \infty & \text{if } x = 1 \\ x + \frac{1}{255} & \text{if } x < 1 \end{cases} \quad \delta_-(x) = \begin{cases} -\infty & \text{if } x = -1 \\ x - \frac{1}{255} & \text{if } x > -1 \end{cases}$$

1. scailing

- 0~255 값을 [-1, 1] 범위로 바꾸면서 모델이 입력을 정규분포랑 잘 맞춰 처리할 수 있음

2. 마지막 단계(Decoder 역할)

- 픽셀 데이터는 discrete이므로 continuous distribution(Gaussian)의 적분 구간 $(\delta_-(x), \delta_+(x))$ 을 설정해 이산화 처리

⇒ 즉, "정수값"을 그대로 쓰는 게 아니라 그 값 주변의 작은 구간 확률로 처리하는 것

3. lossless codelength

- 데이터에 인위적으로 노이즈를 더하거나 복잡한 수학적 보정 안 해도 됨

⇒ 그래서 모델이 학습하는 ELBO는 곧 픽셀 데이터를 압축했을 때의 최적 codelength가 됨

4. 최종 출력

- sampling 다 끝나면 그냥 평균값을 그대로 픽셀로 출력 → 깔끔한 이미지가 나옴

3.4 Simplified training objective

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right] \quad (14)$$

- 앞에서 정의한 reverse process와 decoder를 사용하면, variational bound는 (12), (13)에서 유도된 항들로 구성되며, θ 에 대해 명확하게 미분 가능하고 학습에 바로 사용할 수 있음

4. Experiments

** 여기는 ppt 설명이 너무 완벽해서 발제 ppt 캡처로 대체...

Setting :

We set $T = 1000$ for all experiments so that the number of neural network evaluations needed during sampling matches previous work [53, 55]. We set the forward process variances to constants increasing linearly from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$. These constants were chosen to be small relative to data scaled to $[-1, 1]$, ensuring that reverse and forward processes have approximately the same functional form while keeping the signal-to-noise ratio at $\mathbf{x}_T$ as small as possible ($L_T = D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})) \approx 10^{-5}$ bits per dimension in our experiments).

To represent the reverse process, we use a U-Net backbone similar to an unmasked PixelCNN++ [52, 48] with group normalization throughout [66]. Parameters are shared across time, which is specified to the network using the Transformer sinusoidal position embedding [60]. We use self-attention at the 16×16 feature map resolution [63, 60]. Details are in Appendix B.

- $T = 1000$
- $\beta_t = 0.02$ 까지 선형적으로 증가 ($[-1, 1]$ scaling에 비해 충분히 작은 값)
 - reverse = forward form
- 마지막 단계에서 SNR를 최대한 작게 유지
 - 거의 순수한 Gaussian 상태($\sim \mathcal{N}(\mathbf{0}, \mathbf{I})$)를 만들기 위함
- 신경망은 U-Net

4.1 Sample Quality

Table 1: CIFAR10 results. NLL measured in bits/dim.

Model	IS	FID	NLL Test (Train)
Conditional			
EBM [11]	8.30	37.9	
JEM [17]	8.76	38.4	
BigGAN [3]	9.22	14.73	
StyleGAN2 + ADA (v1) [29]	10.06	2.67	
Unconditional			
Diffusion (original) [53]			≤ 5.40
Gated PixelCNN [59]	4.60	65.93	3.03 (2.90)
Sparse Transformer [7]			2.80
PixelIQN [43]	5.29	49.46	
EBM [11]	6.78	38.2	
NCSNv2 [56]		31.75	
NCSN [55]	8.87 ± 0.12	25.32	
SNGAN [39]	8.22 ± 0.05	21.7	
SNGAN-DDLS [4]	9.09 ± 0.10	15.42	
StyleGAN2 + ADA (v1) [29]	9.74 ± 0.05	3.26	
Ours (L , fixed isotropic Σ)	7.67 ± 0.13	13.51	≤ 3.70 (3.69)
Ours (L_{simple})	9.46 ± 0.11	3.17	≤ 3.75 (3.72)

Table 2: Unconditional CIFAR10 reverse process parameterization and training objective ablation. Blank entries were unstable to train and generated poor samples with out-of-range scores.

Objective	IS	FID
$\tilde{\mu}$ prediction (baseline)		
L , learned diagonal Σ	7.28 ± 0.10	23.69
L , fixed isotropic Σ	8.06 ± 0.09	13.22
$\ \tilde{\mu} - \tilde{\mu}_\theta\ ^2$	–	–
ϵ prediction (ours)		
L , learned diagonal Σ	–	–
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \tilde{\epsilon} - \epsilon_\theta\ ^2$ (L_{simple})	9.46 ± 0.11	3.17

training. However, we found it beneficial to sample quality (and simpler to implement) to train on the following variant of the variational bound:

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t) \right\|^2 \right] \quad (14)$$

- unconditional 모델임에도 불구하고 FID 3.17로, 기존 class-conditional 모델보다 뛰어남



Figure 3: LSUN Church samples. FID=7.89

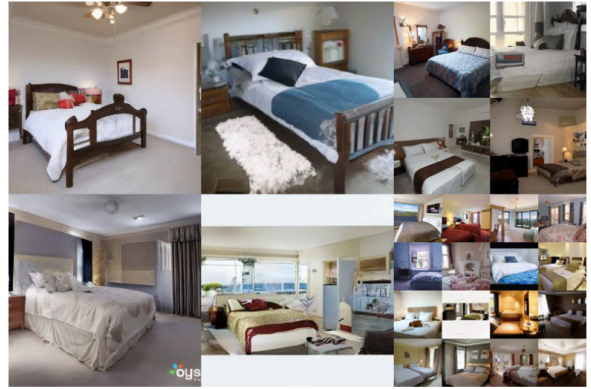


Figure 4: LSUN Bedroom samples. FID=4.90

Algorithm 3 Sending $\mathbf{x}_0$

- 1: Send $\mathbf{x}_T \sim q(\mathbf{x}_T | \mathbf{x}_0)$ using $p(\mathbf{x}_T)$
- 2: **for** $t = T - 1, \dots, 2, 1$ **do**
- 3: Send $\mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_{t+1}, \mathbf{x}_0)$ using $p_\theta(\mathbf{x}_t | \mathbf{x}_{t+1})$
- 4: **end for**
- 5: Send $\mathbf{x}_0$ using $p_\theta(\mathbf{x}_0 | \mathbf{x}_1)$

Algorithm 4 Receiving

- 1: Receive $\mathbf{x}_T$ using $p(\mathbf{x}_T)$
- 2: **for** $t = T - 1, \dots, 1, 0$ **do**
- 3: Receive $\mathbf{x}_t$ using $p_\theta(\mathbf{x}_t | \mathbf{x}_{t+1})$
- 4: **end for**
- 5: **return** $\mathbf{x}_0$

4.2 Reverse process parameterization and training objective ablation

Table 2: Unconditional CIFAR10 reverse process parameterization and training objective ablation. Blank entries were unstable to train and generated poor samples with out-of-range scores.

Objective	IS	FID
$\tilde{\mu}$ prediction (baseline)		
L , learned diagonal Σ	7.28 ± 0.10	23.69
L , fixed isotropic Σ	8.06 ± 0.09	13.22
$\ \tilde{\mu} - \tilde{\mu}_\theta\ ^2$	–	–
ϵ prediction (ours)		
L , learned diagonal Σ	–	–
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \tilde{\epsilon} - \epsilon_\theta\ ^2 (L_{\text{simple}})$	9.46 ± 0.11	3.17

Reverse process form :

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \quad (1)$$

1. $\tilde{\mu}$ 예측

fixed variance를 사용했을 때, 성능이 향상됨
simplified objective 는 성능 매우 악화
(“-” : 성능 매우 불안정/악화)

2. ϵ 예측

much better when trained with our simplified objective.

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t) \right\|^2 \right] \quad (14)$$

4.3 Progressive coding

Table 1: CIFAR10 results. NLL measured in bits/dim.

Model	IS	FID	NLL Test (Train)
Conditional			
EBM [11]	8.30	37.9	
JEM [17]	8.76	38.4	
BigGAN [3]	9.22	14.73	
StyleGAN2 + ADA (v1) [29]	10.06	2.67	
Unconditional			
Diffusion (original) [53]			≤ 5.40
Gated PixelCNN [59]			3.03 (2.90)
Sparse Transformer [7]			2.80
PixelQIN [43]	5.29	49.46	
EBM [11]	6.78	38.2	
NCSNv2 [56]		31.75	
NCSN [55]	8.87 ± 0.12	25.32	
SNGAN [39]	8.22 ± 0.05	21.7	
SNGAN-DDLS [4]	9.09 ± 0.10	15.42	
StyleGAN2 + ADA (v1) [29]	9.74 ± 0.05	3.26	
Ours (L , fixed isotropic Σ)	7.67 ± 0.13	13.51	≤ 3.70 (3.69)
Ours (L_{simple})	9.46 ± 0.11	3.17	≤ 3.75 (3.72)

codelength:

데이터 $\mathbf{x}$ 를 모델 p_θ 로 압축할 때 필요한 평균 비트 수 (단위: bits)

압축 효율 또는 데이터에 대한 모델 적합도를 나타냄

(아까 배운 ELBO는 NLL의 근사치) $\rightarrow L(x) = -\log_2 p_\theta(x)$

NLL (Negative Log-Likelihood):

평균 codelength

NLL이 낮으면, 압축 효율 $\uparrow$

Sparse Transformer : 2.80 bits/dim $\rightarrow$ 효율이 매우 좋음

Ours : 3.72 bits/dim

Diffusion 모델은 FID는 매우 좋지만,

NLL은 Sparse Transformer보다 높음

즉, 압축 성능은 약간 부족하지만, 이미지 품질은 훨씬 뛰어나다는 뜻

4.3 Progressive coding

Table 1 also shows the codelengths of our CIFAR10 models. The gap between train and test is at most 0.03 bits per dimension, which is comparable to the gaps reported with other likelihood-based models and indicates that our diffusion model is not overfitting (see Appendix D for nearest neighbor visualizations). Still, while our lossless codelengths are better than the large estimates reported for energy based models and score matching using annealed importance sampling [11], they are not competitive with other types of likelihood-based generative models [7].

Since our samples are nonetheless of high quality, we conclude that diffusion models have an inductive bias that makes them excellent lossy compressors. Treating the variational bound terms $L_1 + \dots + L_T$ as rate and L_0 as distortion, our CIFAR10 model with the highest quality samples has a rate of 1.78 bits/dim and a distortion of 1.97 bits/dim, which amounts to a root mean squared error of 0.95 on a scale from 0 to 255. More than half of the lossless codelength describes imperceptible distortions.

Train-Test 간 차이는 최대 0.03 bits/dim에 불과함 → overfitting 되지 않았음

lossless codelength의 절반 이상은 사실상 눈에 보이지 않는 왜곡을 기술하는 데 쓰이고 있음을 알 수 있음
(lossless : 데이터를 완전히 원래대로 복원할 수 있는 압축 vs lossy : 일부 정보를 버리고, 대신 더 효율적으로 압축)

Rate : 데이터 압축 효율
= 1.78 bits/dim

$$\mathcal{L} = L_1 + L_2 + \dots + L_T + L_0$$

Distortion : 왜곡, 재구성 오류
= 1.97 bits/dim

$$\mathcal{L} = (L_1 + \dots + L_T) + L_0$$

즉, ELBO = Rate + Distortion

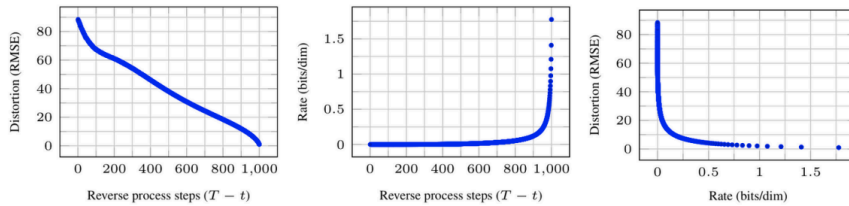
Distortion값을
실제 픽셀 스케일([0,255])로 환산하면
RMSE(root mean squared error) 직접 연결

RMSE $\approx$ 0.95 on a scale from [0, 255]

Progressive lossy compression :

rate-distortion를 탐구하기 위해 다음과 같은 식의 형태를 따르는 손실 압축 코드를 도입함

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_{\theta}(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right] \quad (5)$$



rate가 조금만 늘어나도 distortion이 빠르게 줄어듦

이는 대부분의 비트가 실제로는 imperceptible distortions을 줄이는 데 쓰인다는 것을 보여줌

Progressive generation :

큰 특징 → 먼저 나타남

작은 디테일 → 나중에 나타남

$t = 0$; 원본데이터(x_0)

→ 노이즈 없음

t 가 커질수록 강한 노이즈 추가



Figure 6: Unconditional CIFAR10 progressive generation ($\hat{x}_0$ over time, from left to right). Extended samples and sample quality metrics over time in the appendix (Figs. 10 and 14).



Figure 7: When conditioned on the same latent, CelebA-HQ 256×256 samples share high-level attributes. Bottom-right quadrants are x_t , and other quadrants are samples from $p_{\theta}(x_0 | x_t)$.

Connection to autoregressive decoding :

다음 식은 ELBO를 autoregressive 형태로 다시 쓴 것.

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t \geq 1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right] \quad (5)$$

$$L = D_{\text{KL}}(q(\mathbf{x}_T) \parallel p(\mathbf{x}_T)) + \mathbb{E}_q \left[\sum_{t \geq 1} D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)) \right] + H(\mathbf{x}_0) \quad (16)$$

diffusion 모델을 autoregressive 모델과 연결해서 해석할 수 있음
 하지만 diffusion은 Gaussian noise를 쓰기 때문에 더 부드럽고 강력함
 T는 데이터 차원보다 작아도 되며, 오히려 효율적인 학습과 샘플링 가능
 (T = 1000 vs CIFAR-10 = 32×32×3)

4.4 Interpolation

Interpolation(보간) : 쉽게 말해 A와 B 사이를 이어주는 중간 단계 값
 사진 A(웃는 얼굴)와 사진 B(무표정 얼굴)가 있다면,
 interpolation은 A to B로 서서히 변하는 중간 단계 이미지를 만드는 것

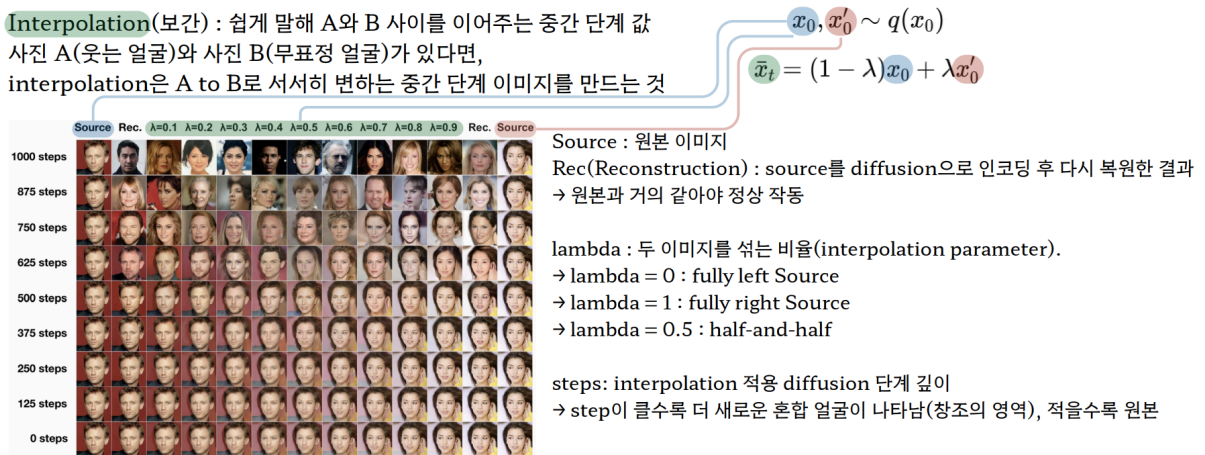


Figure 9: Coarse-to-fine interpolations that vary the number of diffusion steps prior to latent mixing.

5. Related Work

- Flows, VAE 등과 비슷해 보이지만, diffusion은 latent 가 원본 와 독립적이도록 설계 됨
 → 이로써 모델이 점진적으로 [정보 제거 → 복원] 과정을 안정적으로 수행할 수 있음
- ϵ -prediction 방식은 diffusion을 score matching 과 Langevin dynamics sampling에 연결
 → 즉, diffusion은 단순히 노이즈 제거 모델이 아니라, 변분 추론을 통해 학습된

Langevin-like sampler (Langevin dynamics sampling : real-world의 diffusion 현상을 본뜬 sampling 기법)

- 다른 관련 연구; Infusion training, variational walkback, generative stochastic networks 등
비슷하게 markov chain의 전이 규칙을 학습하는 접근
→ 따라서 diffusion은 이들과 같은 맥락에서 발전된 방식
- Energy-based models과의 연결됨
 - 특히 rate-distortion 분석이 annealed importance sampling 방식과 유사
→ Progressive decoding 아이디어는 DRAW, autoregressive 모델의 확장 가능성도 보여줌
(Energy-based model : 데이터를 '에너지 함수'로 표현하고, 좋은 데이터는 낮은 에너지, 나쁜 데이터는 높은 에너지로 학습하는 확률 모델) (엔트로피·자유에너지 개념과 유사한 직관)